

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Шумихина М. А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.10.26

Москва, 2026

Постановка задачи

Вариант 1.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через pipe1, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через pipe2. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Задача 1го Варианта: Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумм.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int pipe(int fd[2])`; – создает однонаправленный канал
- `int dup2(int oldfd, int newfd)` - дублирует дескриптор `oldfd` в позицию `newfd`
- `int execv(const char *path, char *const argv[])` - заменяет образ текущего процесса на новую программу
- `ssize_t read(int fd, void *buf, size_t n)` - читает до `n` байт из дескриптора
- `ssize_t write(int fd, const void *buf, size_t n)` - пишет до `n` байт в дескриптор
- `int open(const char *path, int flags, mode_t mode)` - открывает файл
- `int close(int fd)` - закрывает файловый дескриптор
- `pid_t wait(int *status)` – ожидает завершения дочернего процесса
- `ssize_t readlink(const char *path, char *buf, size_t n)` – возвращает путь, по которому был запущен текущий процесс
- `pid_t getpid(void)` - возвращает PID текущего процесса

Родитель создаёт две трубы: `parent_to_child (pipe1)` и `child_to_parent (pipe2)`. Затем вызывает `fork()`. Дочерний процесс с помощью `dup2` подменяет свой стандартный ввод на read-конец `pipe1`, а стандартный вывод на write-конец `pipe2`, после чего заменяет свой образ на программу `child` через `execv`. Родитель читает с клавиатуры имя файла и строку чисел и записывает их в `pipe1`. Дочерний процесс читает данные из `stdin`, вычисляет сумму и записывает результат в файл через `open`, `write`. Родитель дожидается завершения дочернего процесса через `wait`.

Код программы

parent.c

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
#include <fcntl.h>
```

```
static char CHILD_PROGRAM_NAME[] = "child";
```

```
static int stlen(const char *s) {
```

```
    int n = 0;
```

```
    while (s[n]) ++n;
```

```
    return n;
```

```
}
```

```
static int append(char *dst, int i, const char *src) {
```

```
    int j = 0;
```

```
    while (src[j]) dst[i++] = src[j++];
```

```
    return i;
```

```
}
```

```
static int append_int(char *dst, int i, int x) {
```

```
    char tmp[16];
```

```
    int n = 0;
```

```
    int neg = 0;
```

```
    if (x == 0) { dst[i++] = '0'; return i; }
```

```
    if (x < 0) { neg = 1; x = -x; }
```

```
    while (x > 0) { tmp[n++] = '0' + (x % 10); x /= 10; }
```

```
    if (neg) tmp[n++] = '-';
```

```

for (int a = 0; a < n / 2; ++a) {

    char t = tmp[a];

    tmp[a] = tmp[n - 1 - a];

    tmp[n - 1 - a] = t;

}

for (int a = 0; a < n; ++a) dst[i++] = tmp[a];

return i;

}

int main(int argc, char **argv) {

    (void)argc; (void)argv;

    char prospath[1024];

    {

        ssize_t len = readlink("/proc/self/exe", prospath, sizeof(prospath) - 1);

        if (len == -1) {

            const char msg[] = "error: failed to read full program path\n";

            write(STDERR_FILENO, msg, sizeof(msg) - 1);

            exit(EXIT_FAILURE);

        }

        while (prospath[len] != '/') --len;

        prospath[len] = '\0';

    }

    int parent_to_child[2];

    if (pipe(parent_to_child) == -1) {

        const char msg[] = "error: failed to create pipe1\n";

        write(STDERR_FILENO, msg, sizeof(msg) - 1);

        exit(EXIT_FAILURE);

    }

```

```

int child_to_parent[2];

if (pipe(child_to_parent) == -1) {

    const char msg[] = "error: failed to create pipe2\n";

    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    exit(EXIT_FAILURE);

}


const pid_t child = fork();


switch (child) {

case -1: {

    const char msg[] = "error: failed to spawn new process\n";

    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    exit(EXIT_FAILURE);

} break;


case 0: {

    {

        pid_t pid = getpid();

        char msg[64];

        int i = 0;

        i = append_int(msg, i, pid);

        i = append(msg, i, ": I'm a child\n");

        write(STDOUT_FILENO, msg, i);

    }


    close(parent_to_child[1]);

    close(child_to_parent[0]);


    dup2(parent_to_child[0], STDIN_FILENO);

```

```
close(parent_to_child[0]);
```

```
dup2(child_to_parent[1], STDOUT_FILENO);
```

```
close(child_to_parent[1]);
```

```
{
```

```
    char path[2048];
```

```
    int p = 0;
```

```
    p = append(path, p, progpath);
```

```
    p = append(path, p, "/");
```

```
    p = append(path, p, CHILD_PROGRAM_NAME);
```

```
    path[p] = '\0';
```

```
    char *const args[] = {CHILD_PROGRAM_NAME, NULL};
```

```
    int32_t status = execv(path, args);
```

```
    if (status == -1) {
```

```
        const char msg[] = "error: failed to exec into new executable image\n";
```

```
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
        exit(EXIT_FAILURE);
```

```
    }
```

```
}
```

```
} break;
```

```
default: {
```

```
{
```

```
    pid_t pid = getpid();
```

```
    char msg[128];
```

```
    int i = 0;
```

```
    i = append_int(msg, i, pid);
```

```
    i = append(msg, i, ": I'm a parent, my child has PID ");
```

```
    i = append_int(msg, i, child);

    i = append(msg, i, "\n");

    write(STDOUT_FILENO, msg, i);
}
```

```
close(parent_to_child[0]);

close(child_to_parent[1]);
```

```
char fname[256];

int i = 0;

char c;

while (i < 255) {

    ssize_t r = read(STDIN_FILENO, &c, 1);

    if (r <= 0) break;

    if (c == '\n') break;

    fname[i++] = c;

}

fname[i] = '\0';

if (i == 0) {

    const char msg[] = "error: empty filename\n";

    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    exit(EXIT_FAILURE);

}
```

```
write(parent_to_child[1], fname, strlen(fname));

write(parent_to_child[1], "\n", 1);
```

```
char line[4096];

i = 0;

while (i < 4095) {
```

```

    ssize_t r = read(STDIN_FILENO, &c, 1);

    if (r <= 0) break;

    if (c == '\n') break;

    line[i++] = c;
}

line[i] = '\0';

if (i == 0) {

    const char msg[] = "error: empty numbers line\n";

    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    exit(EXIT_FAILURE);

}

write(parent_to_child[1], line, strlen(line));

write(parent_to_child[1], "\n", 1);

close(parent_to_child[1]);

close(child_to_parent[0]);

wait(NULL);

} break;

}

return 0;

}

```

child.c

```

#include <stdint.h>

#include <stdbool.h>

#include <stdlib.h>

#include <unistd.h>

```

```
#include <fcntl.h>
```

```
static int my_strlen(const char *s) {
```

```
    int n = 0;
```

```
    while (s[n]) ++n;
```

```
    return n;
```

```
}
```

```
static void write_int(int fd, long x) {
```

```
    char tmp[32];
```

```
    int i = 0;
```

```
    int neg = 0;
```

```
    if (x == 0) {
```

```
        write(fd, "0", 1);
```

```
        return;
```

```
    }
```

```
    if (x < 0) { neg = 1; x = -x; }
```

```
    while (x > 0) {
```

```
        tmp[i++] = '0' + (x % 10);
```

```
        x /= 10;
```

```
    }
```

```
    if (neg) tmp[i++] = '-';
```

```
    for (int j = 0; j < i / 2; ++j) {
```

```
        char t = tmp[j];
```

```
        tmp[j] = tmp[i - 1 - j];
```

```
        tmp[i - 1 - j] = t;
```

```
    }
```

```
    write(fd, tmp, i);  
}
```

```
int main(int argc, char **argv) {
```

```
    (void)argc;
```

```
    (void)argv;
```

```
    pid_t pid = getpid();
```

```
    (void)pid;
```

```
    char fname[256];
```

```
    int i = 0;
```

```
    char c;
```

```
    while (i < 255) {
```

```
        ssize_t r = read(STDIN_FILENO, &c, 1);
```

```
        if (r <= 0) break;
```

```
        if (c == '\n') break;
```

```
        fname[i++] = c;
```

```
    }
```

```
    fname[i] = '\0';
```

```
    if (i == 0) {
```

```
        const char msg[] = "error: empty filename\n";
```

```
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
        exit(EXIT_FAILURE);
```

```
    }
```

```
    int32_t file = open(fname, O_WRONLY | O_CREAT | O_TRUNC, 0600);
```

```
    if (file == -1) {
```

```
        const char msg[] = "error: failed to open requested file\n";
```

```

write(STDERR_FILENO, msg, sizeof(msg) - 1);

exit(EXIT_FAILURE);
}

long sum = 0;

long val = 0;

int sign = 1;

int num = 0;

char buf[4096];

ssize_t bytes;

while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {

    for (ssize_t k = 0; k < bytes; ++k) {

        char ch = buf[k];

        if (ch == '-') {

            sign = -1;

            num = 1;

        } else if (ch >= '0' && ch <= '9') {

            val = val * 10 + (ch - '0');

            num = 1;

        } else {

            if (num) {

                sum += sign * val;

                val = 0;

                sign = 1;

                num = 0;

            }

        }

    }

}

```

```

}

if (bytes < 0) {
    const char msg[] = "error: failed to read from stdin\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

if (num) {
    sum += sign * val;
}

write_int(file, sum);
write(file, "\n", 1);

close(file);

(void)my_strlen;

return 0;
}

```

Протокол работы программы

\$./parent

5934: I'm a parent, my child has PID 5935

5935: I'm a child

out.txt

10 20 55

Вывод

В лабораторной я сделала программу, где родитель передаёт числа ребёнку через трубу, а ребёнок считает сумму и пишет в файл. Разобралась, что такое `fork`, `pipe`, `dup2` и `execv`, и как через них связать два процесса. Было тяжело без `printf`, пришлось писать свои функции для вывода чисел.